CSI 2110: Data Structures and Algorithms

Lab 4 – Implementing Sequence ADT and Algorithms

TA: Yifan Qin

University of Ottawa

Fall 2026

Today's Plan

1. **Quick Note $\sim$ 5 min**
2. **Today's Lab $\sim$ 15 min**
3. **Coding Time $\sim$ 60 min**

About Me

- **Name:** Yifan Qin
- **Email:** yqin092@uottawa.ca
- **Office hour:** Monday, 1–2 p.m.
- **Location:** SITE 4035 or via Zoom (Zoom link available on Brightspace)

Quick Note

1. **Thanksgiving: Monday, October 12**
   - There will be **no in-person lab** for Monday's section that week.
   - I will upload a recorded lab video for the week of October 12. You can watch it at home at your convenience.
   - This schedule change applies only to Monday's C02 lab. If you are in another section, please confirm your schedule with your assigned TA.
2. **Lab 3 TA Check Extension**
   - The **Lab 3 TA check deadline** will be extended by **one week** for **all lab sections on Monday**.

Today's Plan

1. **Quick Note $\sim$ 5 min**
2. **Today's Lab $\sim$ 15 min**
3. **Coding Time $\sim$ 60 min**

Today's Lab

- **Task 1:** Implement the Sequence ADT
  - Complete `LinkedSequence` by extending `LinkedPositionalList`.
  - Implement the bridge methods between indices and positions, then the index-based methods.

- **Task 2:** Algorithms for Sequences
  - Complete `isPalindrome` and `inplaceReverse` in `SequenceAlgs.java`.
  - Test your algorithms with `ArraySequence` and `LinkedSequence`.
  - **Optional challenge:** Implement `inplaceKReverse` to reverse each group of k elements.

Getting Started: Lab Files

1. Download and Open the Files

- Download and extract the Lab 4 archive from Brightspace.
- The archive contains the following ten Java files.

Lists	Positional Lists	Sequences	Algorithms & Tests
`List.java`	`Position.java`	`Sequence.java`	`SequenceAlgs.java`
`ArrayList.java`	`PositionalList.java`	`ArraySequence.java`	`SequenceTester.java`
	`LinkedPositionalList.java`	`LinkedSequence.java`	

Before coding: Read the interfaces and the comments in the provided classes.

Getting Started: What to Complete

2. Complete Lab 4

- **Task 1:** Complete `LinkedSequence.java`.
  - Implement the bridge methods: `positionAtIndex` and `indexAtPosition`.
  - Use them with inherited positional-list operations to implement the missing index-based methods.
  - Test your implementation using `SequenceTester.java`.

- **Task 2:** Complete `SequenceAlgs.java`.
  - Implement `isPalindrome` and `inplaceReverse`.
  - **Optional:** Implement `inplaceKReverse`.
  - Test with both `ArraySequence` and `LinkedSequence`.

Note: You can start Task 2 using `ArraySequence`, even if Task 1 is not yet complete.

Task 1: Implementing LinkedSequence

1. Explore the Given Code

- Read `Sequence.java` to understand the required methods.
- Review `LinkedPositionalList`: its positional operations are inherited by `LinkedSequence`.

2. Complete `LinkedSequence`

- `positionAtIndex(i)`: find the position at index `i`.
- `indexAtPosition(p)`: find the index of position `p`.
- Use these bridge methods and inherited operations to implement `get`, `set`, `add`, and `remove` by index.
- Check index bounds: insertion allows $0 \leq i \leq n$; access, replacement, and removal require $0 \leq i < n$.

3. Test Your Implementation

- Run `SequenceTester`.
- Check empty lists, first and last positions, and invalid inputs.

Task 1

```java
public class LinkedSequence<E> extends LinkedPositionalList<E> implements Sequence<E> {

    /**
     * returns position corresponding to index
     * since need to traverse this LinkedPositionalList to get to index i time is O(n)
     * @param i the index or rank of the element in the list (first has index 0, second has ind
     * @return position of the element or null if index does not exist
     * @throws IndexOutOfBoundsException if the index is negative or greater than size()-1
     */
        public Position<E> positionAtIndex(int i) throws IndexOutOfBoundsException {
                checkIndex(i,size()); // checks whether the given index is in the range [0, si

                /******* add your code here **********/

                return null; //dummy return
         }

      /**
        * returns index corresponding to position
        * since need to hop through elements of this LinkedPositionalList to get to position po
        * @param pos - the position or cell you must located in this LinkedPositionalList
        * @return index (also called rank) in the list or -1 if position not found
        *
        */
    public int indexAtPosition(Position<E> pos) {

                /******* add your code here **********/
        return 0; //dummy return
    }

```

Task 1

```
71      /**
72       * Inserts the given element at the specified index of the list, shifting all
73       * subsequent elements in the list one position further to make room
74       * Note: when i=size() it correspond to an addLast when i<size it correspond to addBefore
75       * @param  i   the index at which the new element should be stored
76       * @param  e   the new element to be stored
77       * @throws IndexOutOfBoundsException if the index is negative or greater than size()
78       */
79      public void add(int i, E e){ // insert a new element which will have index i
80          /******* add your code here **********/
81
82      }
83
84      /**
85       * Removes and returns the element at the given index, shifting all subsequent
86       * elements in the list one position closer to the front.
87       * @param  i   the index of the element to be removed
88       * @return the element that had be stored at the given index
89       * @throws IndexOutOfBoundsException if the index is negative or greater than size()
90       */
91      public E remove(int i) throws IndexOutOfBoundsException { // remove element with index i
92
93          /******* add your code here **********/
94
95                  E el=null;    // dummy commands to be removed
96                  return el; // dummy commands to be removed
97      }
98
```

Task 1

```
46      /**
47       * Returns (but does not remove) the element at index i.
48       * @param  i   the index of the element to return
49       * @return the element at the specified index
50       * @throws IndexOutOfBoundsException if the index is negative or greater than size()-1
51       */
52      public E get(int i) throws IndexOutOfBoundsException { // gets element at index i
53
54          /******* add your code here **********/
55          return null;
56      }
57
58      /**
59       * Replaces the element at the specified index, and returns the element previously stored.
60       * @param  i   the index of the element to replace
61       * @param  e   the new element to be stored
62       * @return the previously stored element
63       * @throws IndexOutOfBoundsException if the index is negative or greater than size()-1
64       */
65      public E set(int i, E e) throws IndexOutOfBoundsException { // replaces the element at inde
66
67          /******* add your code here **********/
68          return null;
69      }
70
```

Task 2: Algorithms for Sequences

1. **Complete** `SequenceAlgs.java`
   - `isPalindrome(S)`: check whether the sequence reads the same forwards and backwards.
   - `inplaceReverse(S)`: reverse the sequence by swapping element values.
   - **Optional:** `inplaceKReverse(S, k)`: reverse each group of k elements, including the final shorter group.

2. **Design Your Algorithms**
   - Use operations from the `Sequence` interface so your code works with both implementations.
   - Aim for $O(n)$ time. Repeated index-based access can be costly in `LinkedSequence`.

3. **Test Your Methods**
   - Run `SequenceAlgs` with `ArraySequence` and `LinkedSequence`.
   - Include empty, single-element, odd-length, and even-length sequences.

Task 2: Function you need to finish

```
17      public boolean isPalindrome (Sequence<E> S) {
18
19          // ***** add your code here  ******************
20
21          return true;
22
23      }
24
```

```
35
36      public void inplaceReverse(Sequence<E> S) {
37
38          // ***** add your code here  ******************
39
40      }
41
42      /**
```

```
52      public void inplaceKReverse(Sequence<E> S,int k) {
53
54          // ***** add your code here  ********************
55
56      }
57
```

What to Submit

Submission Package

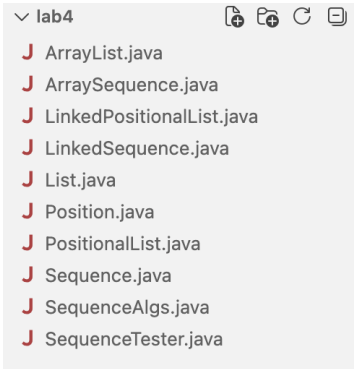
- Package all Java files with a `README` file.
  (Don't change the name of these java files)
- Upload the zip package to **Brightspace**.

Your README Must Include

- Full name(s).
- Student number(s).
- Analysis, if required in this lab.

Submission Deadline: This Thursday at 11:59 p.m.

Submission Package



Today's Plan

1. **Quick Note $\sim$ 5 min**
2. **Today's Lab $\sim$ 15 min**
3. **Coding Time $\sim$ 60 min**

Coding Time

What You Can Do

- Work on your current lab. (Don't forget to read the Lab3Index)
- Find a partner if you would like to work in a pair. (can be different for each lab)
- Demonstrate and run your previous or current lab code to me for grading

Working Together

- You may discuss and complete the lab with your partner.
- Keep your voices not too loud so others can focus.

Finished Early

- Once you have completed all your tasks, you may stay to study or leave early.

Deadline Reminder

Lab 4 Submission Deadline: This Thursday (Oct.8th) 11:59 pm

Written Assignment 1 Deadline: This Friday (Oct. 9th) 11:59 pm

Lab 2 TA Check Deadline: Today's lab

Lab 3 & 4 TA Check Deadline: Next Next Monday's lab (Oct.19th)

References and Acknowledgments

Parts of this presentation are reproduced or adapted from:

- **CSI 2110 lecture materials:** Text, figures, and code examples.
- **Lab materials prepared by other TAs:** Text, figures, and explanations.

Thank you to the course instructors and fellow TAs for sharing their teaching materials.

The End